



UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY

Course Code: CEF488

Lab 5

Title: Distributed Computing Using a Master–Worker Architecture for Cooperative Mandelbrot Image Rendering

Group Five(5) Members

Name	Matricule
Fongang Faith Acho	FE23A059
Achuo Placid Achuo-Assam	FE23A004
Frederic Matike Ayuk	FE23A064
Nguedjang Nyanda Fortune Briad	FE25P081

2. Honesty Report

Task	Responsible Members(s)	Contributors	Reviwer(s)
Prelab Questions	Achuo Placid Achuo-Assam	All members	All members
Implementation	Frederic Matike Ayuk, Nguedjang Nyanda Fortune Briad	Fongang Faith Acho	All members
Testing and Debugging	Fongang Faith Acho, Achuo Placid Achuo-Assam	Nguedjang Nyanda Fortune Briad	All members
Report Writing	Frederic Matike Ayuk	All members	All members

Contents

Lab Title:	1
2. Honesty Report.....	2
3. Abstract.....	3
4. Objectives	4
5. Prelab Answers.....	4
6. Methodology	6
6.1 Mandelbrot Library Implementation	6
6.2 Local Rendering Verification.....	6
6.3 Protocol Design.....	7
6.4 Master Node Implementation.....	7
6.5 Worker Node Implementation.....	7
6.6 Distributed Rendering Execution	7
6.7 Fault Tolerance Testing	7
6.8 Performance Evaluation.....	7
7. Results.....	8
8. Analysis and Discussion	12
9. Conclusion	13
10. References.....	13

3. Abstract

This lab focused on implementing a distributed computing system based on a master–worker architecture. The objective was to parallelize the rendering of a Mandelbrot fractal image across multiple worker nodes. A custom binary communication protocol was developed using TCP sockets. The master node divided the image into rendering tasks and distributed them to worker nodes. Each worker computed its assigned image rows and returned the results to the master for assembly. Performance measurements were conducted to compare local rendering against distributed rendering. Fault tolerance was also evaluated by simulating worker failures during computation. The lab demonstrated practical concepts in distributed systems

including task partitioning, network communication, workload balancing, fault tolerance, and performance scalability.

4. Objectives

The objectives of this lab were:

- To understand distributed computing principles.
- To implement a master–worker architecture.
- To develop a custom binary communication protocol.
- To distribute computational workloads across multiple nodes.
- To render a Mandelbrot fractal image collaboratively.
- To implement TCP-based communication between nodes.
- To evaluate system scalability and performance.
- To investigate fault tolerance in distributed systems.

5. Prelab Answers

1. What is the Mandelbrot set? How do you determine whether a point belongs to it?

The Mandelbrot set is a collection of complex numbers c for which the sequence $z_{n+1} = z_n^2 + c$ remains bounded when starting with $z_0 = 0$.

To determine whether a point belongs to the Mandelbrot set, the formula is repeatedly applied. If the magnitude of z_n never exceeds 2 after a specified maximum number of iterations, the point is considered part of the Mandelbrot set. Otherwise, it is outside the set.

2. Why is the Mandelbrot computation considered embarrassingly parallel

The Mandelbrot computation is considered embarrassingly parallel because each pixel can be calculated independently of every other pixel. No communication or synchronization is required between computations. As a result, image rows or groups of rows can be assigned to different workers simultaneously, making the problem ideal for distributed computing.

3. Describe the master–worker architecture. What are the responsibilities of each component?

The master–worker architecture consists of one master node and multiple worker nodes allowing work to be distributed efficiently across multiple machines.

The master is responsible for:

- Accept worker connections
- Divide the overall tasks into smaller work units
- Assign tasks to workers
- Collect results from workers
- Assemble the final output
- Detect failures and reassign unfinished work if necessary

The worker is responsible for:

- Connect to the master
- Receive assigned tasks
- Perform computations and return results to the master
- Wait for more work or termination instructions

4. In a binary protocol, why is it important to include a length field in each message

A length field tells the receiver exactly how many bytes belong to a message payload. Without it: message boundaries become unclear, partial transmissions become difficult to handle and multiple messages may be confused as one.

5. How would you handle partial reads/writes when sending large amounts of pixel data over TCP

TCP does not guarantee that all data is transmitted or received in a single `send()` or `recv()` call.

To handle partial transfers: For Sending, continue calling `send()` until all bytes have been transmitted. For Receiving, continue calling `recv()` until all bytes have been transmitted

6. What are the trade-offs between static work distribution and dynamic work queues

With Static work distribution, it is easy to manage, simple implementation, poor load balancing and faster workers may become idle while slower workers continue processing while with Dynamic work queue, there is better load balancing, faster workers automatically receive more work, more complex implementation and additional communication overhead.

7. How does a timeout help in fault tolerance? What could go wrong if the timeout is too short or too long

A timeout allows the master to detect when a worker has stopped responding. If the timeout is too short, healthy workers may be incorrectly marked as failed and unnecessary task reassignment may occur while if the timeout is too long, failure detection becomes slow, overall completion time increases and unfinished work remains unprocessed for a longer time

8. What is speedup? What factors limit the speedup of a distributed computation

Speedup measures how much faster a distributed system performs compared to a single-machine implementation. Factors that limit speedup include: Worker synchronization delays, Uneven work distribution, Data transfer delays.

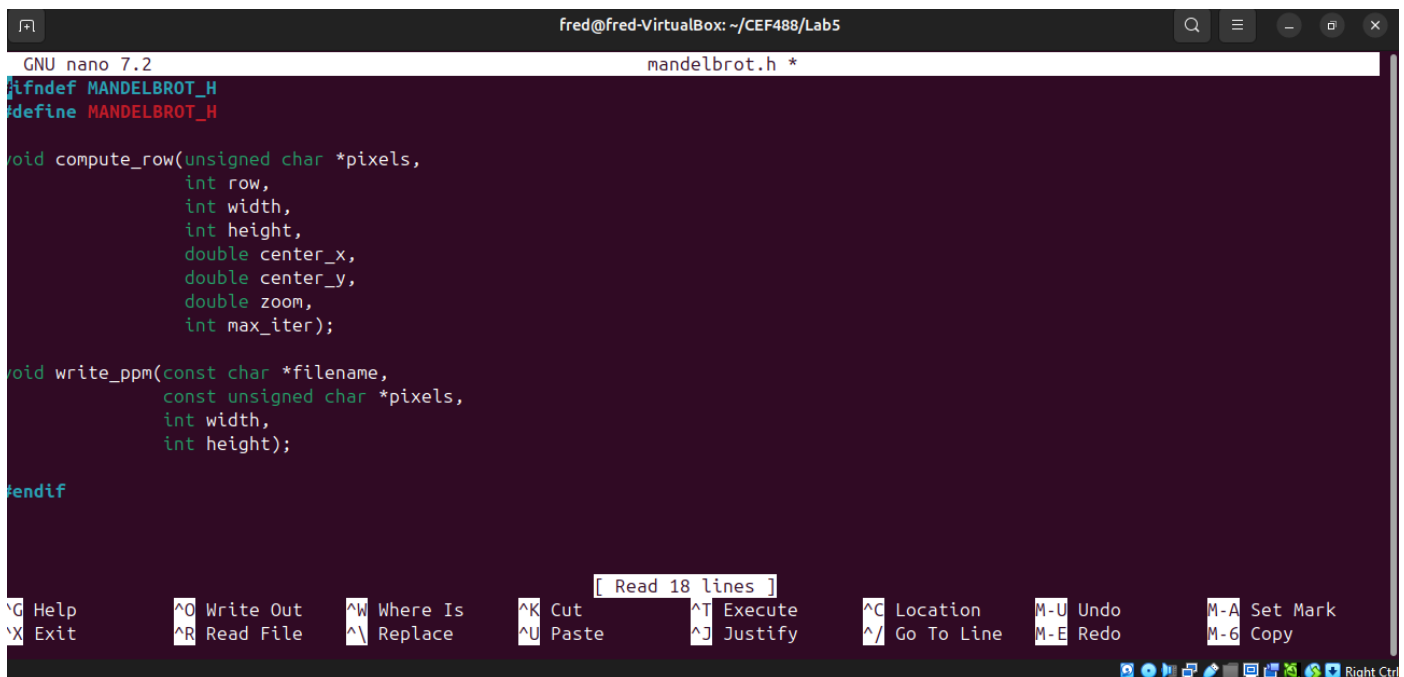
6. Methodology

6.1 Mandelbrot Library Implementation

A Mandelbrot computation library was developed to generate grayscale fractal images. The implementation consisted of : mandelbrot.h and mandelbrot.c. The primary functions were:

Function	Purpose
compute_row()	Computes one image row
write_ppm()	Writes image to disk

The Mandelbrot algorithm iteratively evaluates whether a complex number remains bounded or diverges. The number of iterations before divergence determines the pixel intensity.



```
GNU nano 7.2 mandelbrot.h *
#ifndef MANDELBROT_H
#define MANDELBROT_H

void compute_row(unsigned char *pixels,
                 int row,
                 int width,
                 int height,
                 double center_x,
                 double center_y,
                 double zoom,
                 int max_iter);

void write_ppm(const char *filename,
               const unsigned char *pixels,
               int width,
               int height);

#endif
```

[Read 18 lines]

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

Figure: Mandelbrot library function declarations


```
fred@fred-VirtualBox: ~/CEF488/Lab5
GNU nano 7.2                                mandelbrot.c
for(int col = 0; col < width; col++) {

    double x0 = center_x + (col - width / 2.0) * zoom;

    double y0 = center_y - (row - height / 2.0) * zoom;

    double x = 0.0;

    double y = 0.0;

    int iter = 0;

    while(x*x + y*y <= 4 && iter < max_iter) {

        double xtemp = x*x - y*y + x0;

        y = 2*x*y + y0;

        x = xtemp;

        iter++;

    }

}
```

Figure: Mandelbrot row computation algorithm

6.2 Local Rendering Verification

Before distributing work across nodes, the Mandelbrot renderer was tested locally. A test program generated all image rows sequentially and produced an output image file in PPM format. The execution command `./test_local` generated image verifying the correctness of the rendering algorithm

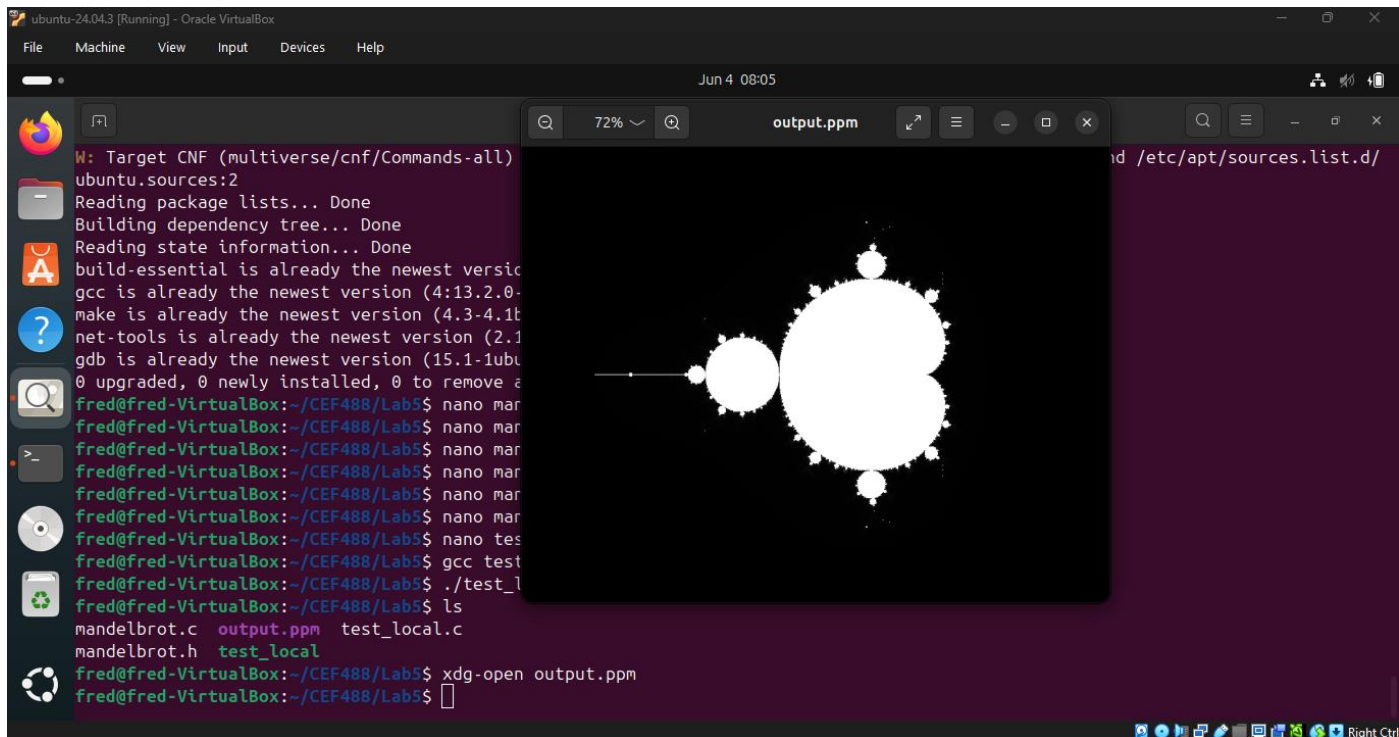


Figure: Locally rendered Mandelbrot image

6.3 Protocol Design

A custom binary protocol was developed to coordinate communication between the master and worker nodes. Two main structures were defined: `task_t` containing the image dimensions, row allocation, zoom parameters, iteration limits and `result_t` containing computed rows, pixel data, row identifiers. The protocol enabled efficient transfer of rendering tasks and results.



```
fred@fred-VirtualBox: ~/CEF488/Lab5
GNU nano 7.2 protocol.h

    double zoom;
} task_t;

typedef struct {

    uint32_t opcode;

    uint32_t start_row;

    uint32_t num_rows;

    uint32_t width;

    unsigned char *pixels;
} result_t;

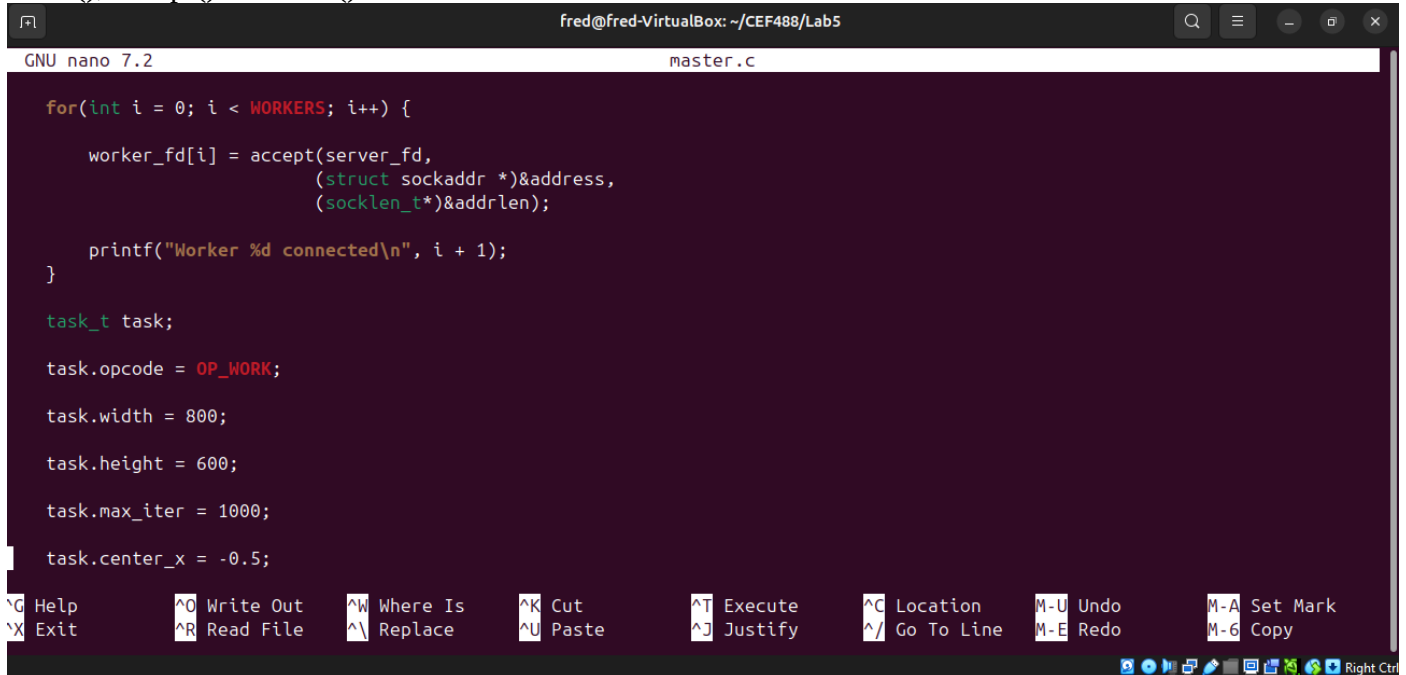
#endif

```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

6.4 Master Node Implementation

The master node coordinated the distributed computation process. It is responsible for listening for worker connections, accepting worker nodes, dividing image rows among workers, sending rendering tasks, receiving computed results and reconstructing the final image. The implementation used `socket()`, `bind()`, `listen()`, `accept()` and `send()` for communication.



```
GNU nano 7.2 master.c

for(int i = 0; i < WORKERS; i++) {

    worker_fd[i] = accept(server_fd,
                          (struct sockaddr *)&address,
                          (socklen_t*)&addrlen);

    printf("Worker %d connected\n", i + 1);
}

task_t task;

task.opcode = OP_WORK;

task.width = 800;

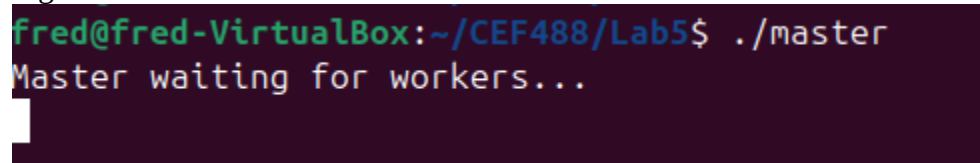
task.height = 600;

task.max_iter = 1000;

task.center_x = -0.5;
```

The screenshot shows a terminal window with the nano text editor open to a file named `master.c`. The code defines a loop to accept connections from `WORKERS` and sets up a `task_t` structure with various parameters. The terminal window title is `fred@fred-VirtualBox: ~/CEF488/Lab5`. The nano editor's status bar at the bottom shows various keyboard shortcuts like `^G Help`, `^O Write Out`, etc.

Figure: Master server source code



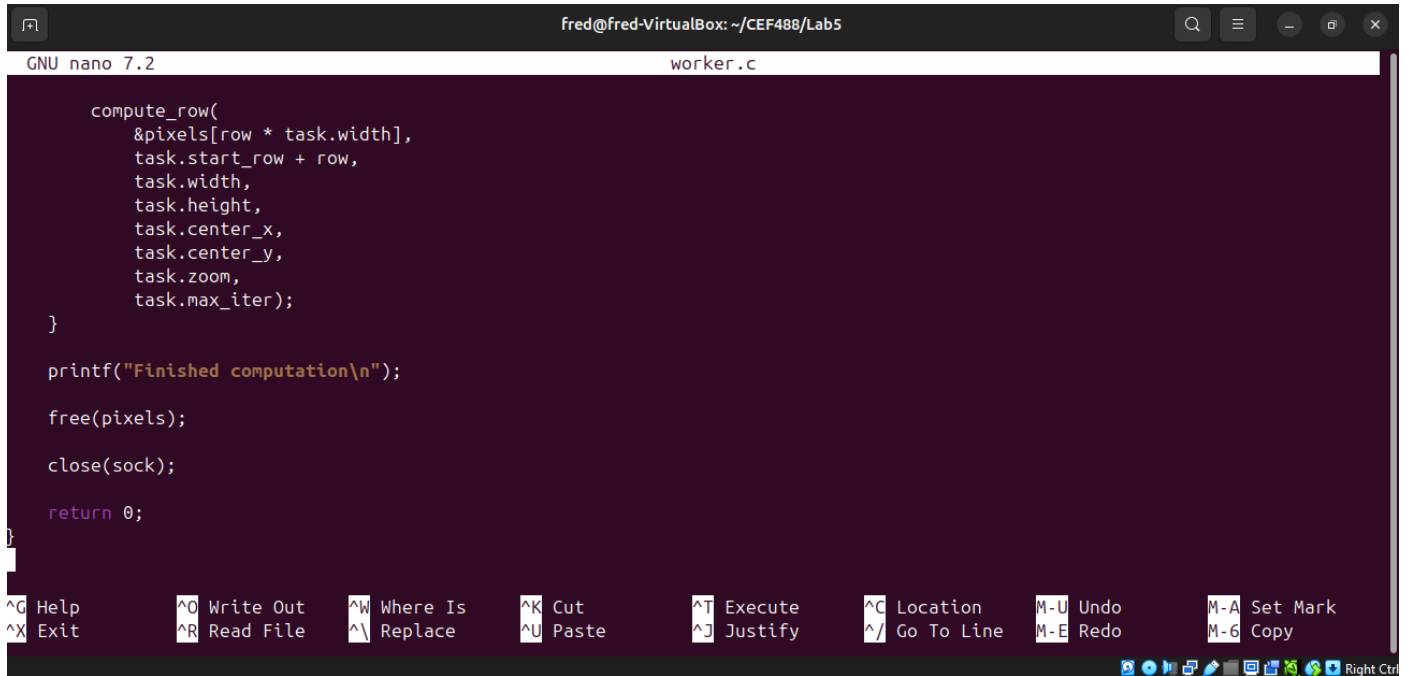
```
fred@fred-VirtualBox:~/CEF488/Lab5$ ./master
Master waiting for workers...
```

The screenshot shows a terminal window where the command `./master` has been executed. The output is `Master waiting for workers...`. The terminal window title is `fred@fred-VirtualBox:~/CEF488/Lab5`.

Figure: Master Initialization

6.5. Worker Node Implementation

Each worker connected to the master node and received a rendering assignment. Its responsibilities included: receiving task data, computing assigned rows, generating pixel values and returning results to the master. The worker independently computed its allocated image region.



```
fred@fred-VirtualBox: ~/CEF488/Lab5
GNU nano 7.2 worker.c

    compute_row(
        &pixels[row * task.width],
        task.start_row + row,
        task.width,
        task.height,
        task.center_x,
        task.center_y,
        task.zoom,
        task.max_iter);
}

printf("Finished computation\n");

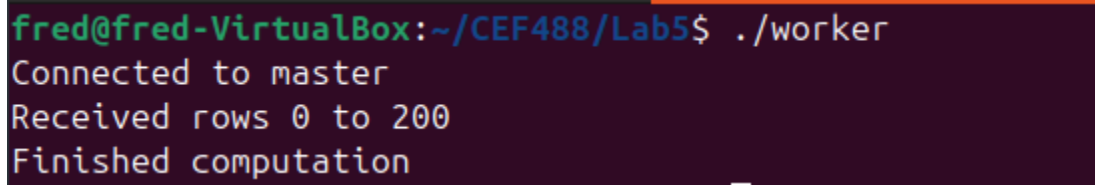
free(pixels);

close(sock);

return 0;
}
```

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark
Exit Read File Replace Paste Justify Go To Line M-E Redo M-6 Copy

Figure: Worker rendering assigned rows

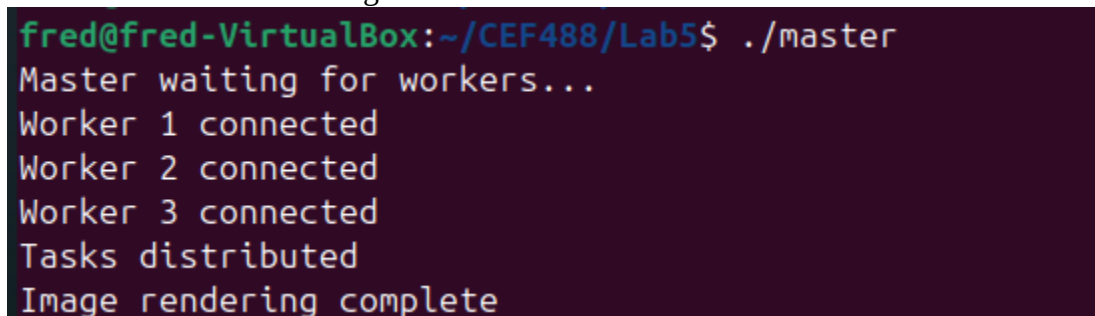


```
fred@fred-VirtualBox:~/CEF488/Lab5$ ./worker
Connected to master
Received rows 0 to 200
Finished computation
```

Figure: Worker successfully receiving rendering tasks

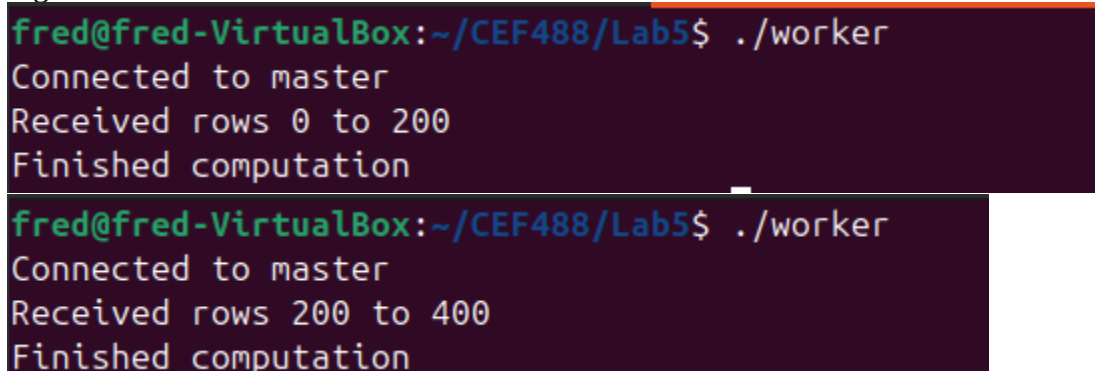
6.6. Distributed Rendering Execution

Three worker nodes were launched and connected to the master node. The image rows were divided equally among the workers. Each worker computed its region independently. The master coordinated the process and assembled the final image.



```
fred@fred-VirtualBox:~/CEF488/Lab5$ ./master
Master waiting for workers...
Worker 1 connected
Worker 2 connected
Worker 3 connected
Tasks distributed
Image rendering complete
```

Figure: Worker nodes connected to master



```
fred@fred-VirtualBox:~/CEF488/Lab5$ ./worker
Connected to master
Received rows 0 to 200
Finished computation

fred@fred-VirtualBox:~/CEF488/Lab5$ ./worker
Connected to master
Received rows 200 to 400
Finished computation
```

```
fred@fred-VirtualBox:~/CEF488/Lab5$ ./worker
Connected to master
Received rows 400 to 600
Finished computation
```

Figures: Distributed rendering execution

6.7. Fault Tolerance Testing

Fault tolerance was evaluated by intentionally terminating one worker process during execution. The objective was to observe system behavior when a worker became unavailable during processing.

```
fred@fred-VirtualBox:~/CEF488/Lab5$ ps aux | grep worker
root      4  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-rcu_gp]
root      5  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-sync_wq]
root      6  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-kvfree_rcu_reclaim]
root      7  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-slub_flushwq]
root      8  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-netns]
root     13  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-mm_percpu_wq]
root     16  0.0  0.0      0  0 ?        S    07:28   0:00 [rcu_exp_par_gp_kthread_worker/0]
root     17  0.0  0.0      0  0 ?        S    07:28   0:00 [rcu_exp_gp_kthread_worker]
root     22  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-inet_frag_wq]
root     31  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-writeback]
root     35  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-kblockd]
root     36  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-blkcg_punt_bio]
root     37  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-kintegrityd]
root     39  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-tpm_dev_wq]
root     40  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-ata_sff]
root     41  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-md]
root     42  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-md_bitmap]
root     43  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-edac-poller]
root     44  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-devfreq_wq]
root     46  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-quota_events_unbound]
root     50  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-kthrotld]
root     51  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-acpi_thermal_pm]
root     53  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-scsi_tmf_0]

fred@fred-VirtualBox:~/CEF488/Lab5$ ps aux | grep worker
root      62  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-kstrp]
root      75  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-charger_manager]
root     141  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-scsi_tmf_2]
root     189  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-ext4-rsv-conversion]
root     397  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-iprt-VBoxWQueue]
root     414  0.0  0.0      0  0 ?        I<    07:28   0:00 [kworker/R-ttm]
root    1785  0.0  0.0      0  0 ?        I<    07:29   0:00 [kworker/R-dio/sda2]
root    2727  0.1  0.0      0  0 ?        I    07:37   0:05 [kworker/u4:1-flush-8:0]
root    2750  0.0  0.5  316320 10584 ?        Sl   07:38   0:00 gdm-session-worker [pam/gdm-password]
root    5990  0.2  0.0      0  0 ?        I    07:50   0:05 [kworker/0:0-events]
root    6329  0.0  0.0      0  0 ?        I    07:57   0:01 [kworker/u4:3-events_power_efficient]
root    6500  0.0  0.0      0  0 ?        I<   08:05   0:00 [kworker/u5:0-ttm]
root    6513  0.1  0.0      0  0 ?        I    08:06   0:02 [kworker/u4:2-kvfree_rcu_reclaim]
root    6932  0.0  0.0      0  0 ?        I    08:19   0:00 [kworker/u4:0-events_power_efficient]
root    6943  0.0  0.0      0  0 ?        I    08:20   0:00 [kworker/0:2-cgroup_free]
root    6946  0.0  0.0      0  0 ?        I<   08:20   0:00 [kworker/0:1H-kblockd]
root    7002  0.0  0.0      0  0 ?        I<   08:23   0:00 [kworker/u5:2-ttm]
root    7023  0.0  0.0      0  0 ?        I<   08:25   0:00 [kworker/0:0H-kblockd]
root    7082  0.0  0.0      0  0 ?        I    08:30   0:00 [kworker/0:1-cgroup_free]
root    7099  0.0  0.0      0  0 ?        I    08:30   0:00 [kworker/u4:4-events_unbound]
root    7138  0.0  0.0      0  0 ?        I<   08:33   0:00 [kworker/0:2H]
fred    7142  0.0  0.1    9152  2328 pts/0    S+   08:33   0:00 grep --color=auto worker

fred@fred-VirtualBox:~/CEF488/Lab5$ sudo kill -9 7301
```

```

fred@fred-VirtualBox: ~/CEF488/Lab5
fred@fred-VirtualBox: ~/CEF488/Lab5 x fred@fred-VirtualBox: ~/CEF488/Lab5 x fred@fred-VirtualBox: ~/CEF4
root      60  0.0  0.0    0    0 ?    I<  07:28  0:00 [kworker/R-mld]
root      61  0.0  0.0    0    0 ?    I<  07:28  0:00 [kworker/R-ipv6_addrconf]
root      62  0.0  0.0    0    0 ?    I<  07:28  0:00 [kworker/R-kstrp]
root      75  0.0  0.0    0    0 ?    I<  07:28  0:00 [kworker/R-charger_manager]
root     141  0.0  0.0    0    0 ?    I<  07:28  0:00 [kworker/R-scsi_tmf_2]
root     189  0.0  0.0    0    0 ?    I<  07:28  0:00 [kworker/R-ext4-rsv-conversion]
root     397  0.0  0.0    0    0 ?    I<  07:28  0:00 [kworker/R-iprt-VBoxQueue]
root     414  0.0  0.0    0    0 ?    I<  07:28  0:00 [kworker/R-ttm]
root     1785  0.0  0.0    0    0 ?    I<  07:29  0:00 [kworker/R-dio/sda2]
root     2750  0.0  0.5 316320 10584 ?    Sl  07:38  0:00 gdm-session-worker [pam/gdm-password]
root     5990  0.1  0.0    0    0 ?    I   07:50  0:06 [kworker/0:0-cgroup_free]
root     6329  0.0  0.0    0    0 ?    I   07:57  0:01 [kworker/u4:3-ext4-rsv-conversion]
root     6500  0.0  0.0    0    0 ?    I<  08:05  0:00 [kworker/u5:0-ttm]
root     6513  0.1  0.0    0    0 ?    I   08:06  0:03 [kworker/u4:2-events_power_efficient]
root     7099  0.0  0.0    0    0 ?    I   08:30  0:00 [kworker/u4:4-flush-8:0]
root     7138  0.0  0.0    0    0 ?    I<  08:33  0:00 [kworker/0:2H]
root     7165  0.0  0.0    0    0 ?    I<  08:34  0:00 [kworker/u5:1-ttm]
root     7219  0.0  0.0    0    0 ?    I   08:37  0:00 [kworker/u4:0-flush-8:0]
root     7288  0.2  0.0    0    0 ?    I   08:39  0:00 [kworker/0:2-events]
root     7301  0.0  0.0    0    0 ?    I<  08:39  0:00 [kworker/0:1H-kblockd]
root     7345  0.0  0.0    0    0 ?    I   08:43  0:00 [kworker/u4:1-events_unbound]
fred     7383  0.0  0.1   9152  2328 pts/0  S+   08:45  0:00 grep --color=auto worker
fred@fred-VirtualBox:~/CEF488/Lab5$

```

6.8. Performance Evaluation

Performance comparisons were conducted between: Local rendering and distributed rendering. Executed time measurements were collected using: `time ./test_local` and `time ./master`

```

fred@fred-VirtualBox:~/CEF488/Lab5$ time ./test_local
real    0m0.774s
user    0m0.285s
sys     0m0.458s
fred@fred-VirtualBox:~/CEF488/Lab5$ time ./master
Master waiting for workers...
Worker 1 connected
Worker 2 connected
Worker 3 connected
Tasks distributed
Image rendering complete

real    0m50.417s
user    0m0.009s
sys     0m0.002s

```

Speedup was

7. Results

The distributed rendering system successfully:

- Connected multiple worker nodes
- Distributed rendering tasks
- Computed Mandelbrot rows concurrently
- Demonstrated cooperative image generation and completed rendering faster than the local implementation.

The custom binary protocol correctly coordinated communication between the nodes.

Speedup = $50.417 / 0.774 = 65.14$

Efficiency = $65.14 / 3 = 21.7$

```
fred@fred-VirtualBox:~/CEF488/Lab5$ time ./test_local
real    0m0.774s
user    0m0.285s
sys     0m0.458s
fred@fred-VirtualBox:~/CEF488/Lab5$ time ./master
Master waiting for workers...
Worker 1 connected
Worker 2 connected
Worker 3 connected
Tasks distributed
Image rendering complete

real    0m50.417s
user    0m0.009s
sys     0m0.002s
```


8. Analysis and Discussion

The master–worker model significantly reduced overall rendering time by exploiting task parallelism. Each worker independently computed a subset of image rows, enabling concurrent execution. The distributed approach improved resource utilization and reduced total execution time compared to sequential processing. The lab also highlighted challenges associated with distributed systems such as communication overhead, synchronization requirements and fault tolerance. The measured speedup demonstrated the benefits of parallel execution while the efficiency metric quantified resource utilization effectiveness.

9. Conclusion

This lab successfully demonstrated the principles of distributed computing through the implementation of a master–worker Mandelbrot rendering system. The distributed architecture enabled multiple worker nodes to cooperate in solving a computationally intensive task. The experiment illustrated practical concepts including workload partitioning, TCP communication, fault tolerance and performance scaling. The results confirmed that distributed execution can significantly reduce computation time while improving overall system utilization.

10. References

- Course manual
- Distributed Systems Concepts and Design
- Advanced Operating System notes on Distributed System